

Estate Manager — Deployment Guide

How to put the system online so it can be demonstrated **without running it on your own machine** — free, single-service hosting on **Render**.

Related documents

- [USER MANUAL.md](#) — day-to-day usage of the system.
- [CONFIGURATION.md](#) — starting the app locally, accounts, integrations.
- [TROUBLESHOOTING.md](#) — fixing common problems.

How it works

The app is two processes in development (Vite web app + API). For deployment it is **one process**: the API server (`server/index.js`) also serves the pre-built web app from `client/dist` . This is automatic — when a build exists, the server serves it and falls back to `index.html` for browser routes (`server/index.js` → *Production static hosting*).

So a single Render **Web Service** runs the whole system.

1. Push the project to GitHub (public)

```
cd /Users/richardmuia/Documents/estatemanager

# one-time, if not already done
git init
git branch -M main
git add -A
git commit -m "Estate Manager demo"

# create a public repo and push (GitHub CLI)
gh repo create estate-manager --public --source=. --push

# or without the CLI: create an empty repo at https://github.com/new, then:
# git remote add origin https://github.com/<you>/estate-manager.git
# git push -u origin main
```

`node_modules/` , `dist/` and `server/data/` (the SQLite files) are already gitignored — only source code is committed.

GitHub push auth (known gotcha): GitHub no longer accepts a password over `git push https://...` . You'll get "Invalid username or token. Password authentication is not supported for Git operations." Fix it with the GitHub CLI (one-time):

```
gh auth login          # browser sign-in
gh auth setup-git      # make git use your token
git push -u origin main
```

(Alternatively add an SSH key at github.com/settings/keys and `git remote set-url origin git@github.com:<you>/estatemanager.git`.)

2. Create the free web service

1. Go to <https://dashboard.render.com> → **New** → **Web Service**.
2. Paste your **public repo URL** (no GitHub sign-in needed for a public repo) — or connect your GitHub account and pick the repo.
3. Give it a name, e.g. `estate-manager-demo`.
4. Set these values:

Setting	Value
Runtime	Node 18 (pinned in <code>package.json</code> via <code>engines</code> — required for <code>better-sqlite3</code> prebuilt binaries)
Build command	leave the default <code>npm install; npm run build</code> — the root <code>build</code> script installs client deps (incl. <code>vite</code>) and builds automatically
Start command	<code>node server/index.js</code>
Region	nearest to your audience
Instance type	Free

5. Click **Create Web Service** and wait for the build (≈2–4 minutes).

Build gotchas (already handled): Render's default build sets `NODE_ENV=production`, which makes `npm install` skip devDependencies. The root `build` script therefore installs server **and** client deps with `--include=dev` and then runs `vite build` (`vite` is also a runtime dependency of the client). If a build still fails after a change, use **Manual Deploy** → **Clear build cache & deploy** once to clear stale caches.

3. Access the live system

- URL: `https://estatemanager-wb32.onrender.com` (Render assigns each service a random suffix — copy the exact URL from the Render **dashboard** → **Services** page → **Settings** → **Public URL**)
 - Login: `admin` / `admin123`
 - The database is created and **auto-seeded** with demo data (3 properties, 48 units, tenants, leases, payments) on the very first start — nothing to do.
-

4. Sharing for a demo

Share the Render URL and walk people through it. Good demo flow:

1. **Dashboard** — overview cards, recent payments, renewals due.
2. **Onboarding** — move a tenant into a unit (creates tenant + lease in one step).
3. **Rent Payments** — record a payment; open the **rent ledger**.
4. **Reports** — weekly/monthly summaries.
5. **Lease Renewals** — expire/renew a lease (+10%), send a notice.
6. **Notifications** — reminders are recorded as **simulated** (no SMS/email credentials) so everyone can see the flow safely.
7. **Activity** — the audit trail showing everything you just did.

Demoing on a phone? The deployed app is a **PWA** — on a phone the sidebar becomes a **bottom tab bar** (swipe it sideways to scroll the tabs) and tables stack into labelled cards. For the best look, on iPhone Safari tap **aA** → **Website Settings** and make sure **"Request Desktop Website" is OFF**, then reload — otherwise the phone shows the desktop layout. Optionally Add to Home Screen to run it as a full-screen app.

5. Free-tier caveats to know before demo day

Caveat	Effect	What to do
Sleeps when idle	After ~15 min without traffic the service sleeps; the first request after sleep takes ~50 s to wake up	Warn viewers, or keep someone on the page; open the URL a minute before the demo starts
Ephemeral disk	Files (incl. <code>estate.db</code>) reset on every redeploy ; the DB is re-seeded automatically	Fine for a demo; for persistence use a paid Render instance or a mounted volume (Railway)
Simulated messages	SMS/email stay simulated (no credentials configured)	Intended — nothing real is sent during demos
Free plan limits	One free service per account (3 total historically)	Delete the service when the demo is finished

6. Redeploying after code changes

1. Commit and push to GitHub: `git commit -am "..." && git push`.
2. Render auto-rebuilds from the repo. (If you disabled auto-deploy, hit **Manual Deploy** → **Deploy latest commit**.)

A redeploy wipes the demo data (ephemeral disk). If people made changes during the demo and you want to keep the demo as-is, **don't redeploy** — leave the running service untouched.

7. Alternative hosts

Same single command works for the whole app: `node server/index.js`.

Host	Notes
Railway	Free trial; easier persistence (Volumes mounted at <code>server/data</code>)
Heroku	Requires Node 18 buildpack; ephemeral DB (re-seeds per restart)

Fly.io	Good free allowances, but needs a Dockerfile; ephemeral disk
Local tunnel (demo only)	<code>cloudflared tunnel --url http://localhost:5173</code> — public URL, but your machine must stay on

Avoid **serverless** (Vercel/Netlify Functions, AWS Lambda): SQLite is a file and cannot persist (or even reliably write) in ephemeral, request-scoped filesystems.